University of Colombo School of Computing

SCS 1304 - Problem Solving Strategies and Computation Approaches

Lab Sheet 04

Tracing a Recursive Function with a Trace Table & Trace Tree

01. Consider the following recursive function that calculates the factorial of only odd/even numbers

```
def double_factorial(n):
    if n <= 1:
        return 1
    else:
        return n * double_factorial(n - 2)
```

Use a trace table to trace the execution of ***double_factorial***(8).

Instructions:

I. Create a trace table with columns for the function call, the value of **n**, and the return value.

II. Fill in the table step-by-step as the function is called and returns values.

02. Consider the following recursive function that calculates the sum of the first **n** squares

```
def sum_squares(n):
    if n == 0:
        return 0
    else:
        return n² + sum_squares(n - 1)
```

Draw a trace tree to trace the execution of ***sum_squares***(5).

Instructions:

a. Start with the root node as ***sum_squares***(5).

b. Draw branches for each recursive call until you reach the base case.

c. Label each node with the function call and its return value.

03. A bacteria colony doubles its population every hour. However, once the population reaches 2,000,000, it starts to decline by half every hour due to limited resources. Given an initial population of **P** bacteria, Use the following recursive function to calculate the population after **n** hours.

```
function bacteria_growth(P, n):
  if n == 0:
    return P
  else if P >= 2000000:
    return bacteria_growth(P / 2, n - 1)
  else:
    return bacteria_growth(P * 2, n - 1)
```

I. Create a trace table with columns for the function call, the value of **P**, the value of **n**, and the return value.

Sample table format ;

Function	**P**	**n**	**Return**

II. Fill in the table step-by-step as the function is called and returns values for bacteria_growth(1000, 5).

III. What is the initial population of the bacteria?

IV. What is the condition for the base case in this function?

V. Describe the recursive case when the population is less than 2,000,000.

VI. Describe the recursive case when the population is greater than or equal to 2,000,000.

VII. What value is returned when the function reaches the base case?

VIII. What is the return value for bacteria_growth(1000, 5)?

IX. Modify the function to include a limit where the population can no longer decline (e.g., a minimum population of 10). Provide the pseudo code and trace table for bacteria_growth(1000, 10, 100) where the third parameter is the minimum population limit.

4. A delivery drone starts from a warehouse and travels to deliver packages to various destinations. After each delivery, it returns to the warehouse to pick up the next package. The distance to each destination is given in an array. Use the following recursive function to calculate the total distance traveled by the drone after delivering all the packages.

```
function total_distance(drone_distance, index):
    if index == length(drone_distance):
        return 0
    else:
    return 2 * drone_distance[index] + total_distance(drone_distance, index + 1)
```

I. Trace a tree, start with the root node as total_distance([7, 12, 4], 0).

II. Draw branches for each recursive call until you reach the base case.

III. Label each node with the function call and its return value.

IV. What is the initial list of distances to the destinations?

V. What is the condition for the base case in this function?

VI. Describe the recursive case for calculating the total distance.

VII. What value is returned when the function reaches the base case?

VIII. How many recursive calls are made to calculate the total distance?

IX. What is the total distance traveled by the drone?